

TEMPLATE PARA ENTREGA DO PROJETO DA DISCIPLINA

DevOPs na Prática

Fase 2: Entrega Contínua, Monitoramento e Segurança

NOME DO ESTUDANTE:

CAMILA SILVA GREFF

SEÇÃO 1

Pipeline de Entrega Contínua

a) EXPANSÃO DO PIPELINE DE CI PARA INCLUSÃO DA ENTREGA CONTÍNUA (CD):

O workflow do GitHub Actions (*.github/workflows/ci.yml*) passou a executar etapas de Integração Contínua e Entrega Contínua, incluindo build da aplicação, execução dos testes automatizados, validação dos arquivos Terraform, build da imagem Docker e publicação da imagem no GitHub Container Registry.

O pipeline foi configurado para executar tanto em *pull requests* quanto em *push na branch main*. No desenvolvimento individual deste projeto, a validação e a entrega contínua foram demonstradas por meio de push direto na branch main. Em eventos de *pull_request*, o pipeline executa as validações de qualidade, testes, validação Terraform e build da imagem Docker, sem realizar publicação. Em eventos de push na branch main, além das validações, o pipeline realiza a publicação da imagem Docker no registry, utilizando as tags *latest* e *sha-<commit>*.

Essa expansão permite que a aplicação seja validada, empacotada e disponibilizada de forma padronizada, reduzindo erros manuais e garantindo maior confiabilidade no processo de entrega.

Etapas principais do pipeline:

1. Checkout do código no repositório.
2. Instalação das dependências Node.js.
3. Compilação da aplicação TypeScript.
4. Execução dos testes automatizados.
5. Validação dos arquivos Terraform.
6. Build da imagem Docker.
7. Publicação da imagem no GitHub Container Registry em push na branch main.

SEÇÃO 2

Implementação de Containers e Orquestração

a) CONTAINERIZAÇÃO DA APLICAÇÃO UTILIZANDO DOCKER:

Insira o link de acesso ao repositório (GitHub).

A aplicação foi containerizada utilizando Docker. Para isso, foi criado um arquivo *Dockerfile* na raiz do projeto, responsável por definir o processo de construção da imagem da aplicação, com múltiplos estágios.

No primeiro estágio são instaladas as dependências do projeto, a aplicação TypeScript é compilada e os testes automatizados são executados. No segundo estágio é criada a imagem final de produção, contendo apenas os arquivos necessários para executar a API.

Também foi criado o arquivo *.dockerignore*, com o objetivo de evitar que arquivos desnecessários sejam enviados ao contexto de build, como *node_modules*, logs, arquivos temporários, estados Terraform e variáveis sensíveis.

Comandos principais utilizados:

- `docker build -t devops-fase2-api:local .`
- `docker run --name devops-fase2-api -p 3000:3000 devops-fase2-api:local`

A aplicação fica disponível em: <http://localhost:3000/health>

Link do repositório GitHub: <https://github.com/CamiGreff/devops-fase2-api>

b) SCRIPTS DE DEPLOY USANDO OS CONTAINERS:

Insira o link de acesso ao repositório (GitHub).

O projeto possui scripts para Windows PowerShell e para Linux/macOS:

- *scripts/deploy-local.ps1*
- *scripts/deploy-local.sh*

Esses scripts executam o processo de build da imagem Docker, verificam se já existe um container antigo com o mesmo nome, removem o container anterior quando necessário e iniciam um novo container com a aplicação.

Também foi criado o arquivo *docker-compose.yml*, permitindo a execução da aplicação por meio de orquestração local simples com Docker Compose.

- Comando para deploy com Docker Compose: *docker compose up -d --build*
- Comando para execução do script no Windows: *\scripts\deploy-local.ps1*
- Comando para execução do script no Linux/macOS: *sh scripts/deploy-local.sh*

Após o deploy, a API pode ser testada pelo endpoint: <http://localhost:3000/health>

Os logs do container podem ser consultados com: *docker logs devops-fase2-api*

Link do repositório GitHub: <https://github.com/CamiGreff/devops-fase2-api>

SEÇÃO 3

Relatório Final e Demonstração

a) RELATÓRIO DETALHADO DAS ETAPAS DO PROJETO → FASE 1

Na fase 1, o projeto teve como foco a Configuração e Automação Inicial.

Foi criada uma API simples em Node.js com TypeScript, contendo o endpoint *GET /health*, utilizado para verificar o estado da aplicação.

Também foram implementados testes automatizados para validar o retorno correto do endpoint de saúde e o comportamento de erro em rotas inexistentes.

O código foi versionado em um repositório GitHub, permitindo controle de versão e rastreabilidade das alterações.

O pipeline inicial de Integração Contínua foi configurado com GitHub Actions, para o fim de realizar a instalação das dependências, a compilação da aplicação TypeScript e a execução dos testes automatizados a cada alteração enviada ao repositório.

Além disso, foram criados scripts de Infraestrutura como Código com Terraform, organizados na pasta *infra/*, com o objetivo de demonstrar a definição de infraestrutura de forma versionada e automatizada.

Principais entregas da fase 1:

- API em TypeScript.
- Endpoint GET /health.
- Testes automatizados.
- Repositório GitHub.
- Pipeline de CI com GitHub Actions.
- Validação de build e testes.
- Scripts Terraform para IaC.
- Documentação inicial no README.

a) RELATÓRIO DETALHADO DAS ETAPAS DO PROJETO → FASE 2

Na fase 2, o projeto foi expandido para incluir Entrega Contínua, containerização, deploy com containers, monitoramento básico, logging, testes e segurança.

O pipeline de CI foi ampliado para CI/CD. Além de compilar a aplicação e executar os testes automatizados, o workflow também valida os arquivos Terraform, constrói a imagem Docker e publica essa imagem no GitHub Container Registry quando ocorre um push na branch main.

A aplicação foi containerizada com Docker, utilizando um *Dockerfile* em múltiplos estágios. Também foi criado um arquivo *.dockerignore* para evitar o envio de arquivos desnecessários ou sensíveis ao contexto Docker.

Para a execução da aplicação em containers, foram criados um arquivo *docker-compose.yml* e scripts de deploy local para Windows e Linux/macOS. Esses recursos automatizam a criação da imagem, a remoção de containers antigos e a inicialização de um novo container.

O monitoramento básico foi implementado por meio do *endpoint /health* e de *healthchecks* no ambiente Docker. Os logs da aplicação são enviados para a saída padrão do container e podem ser consultados com o comando *docker logs*.

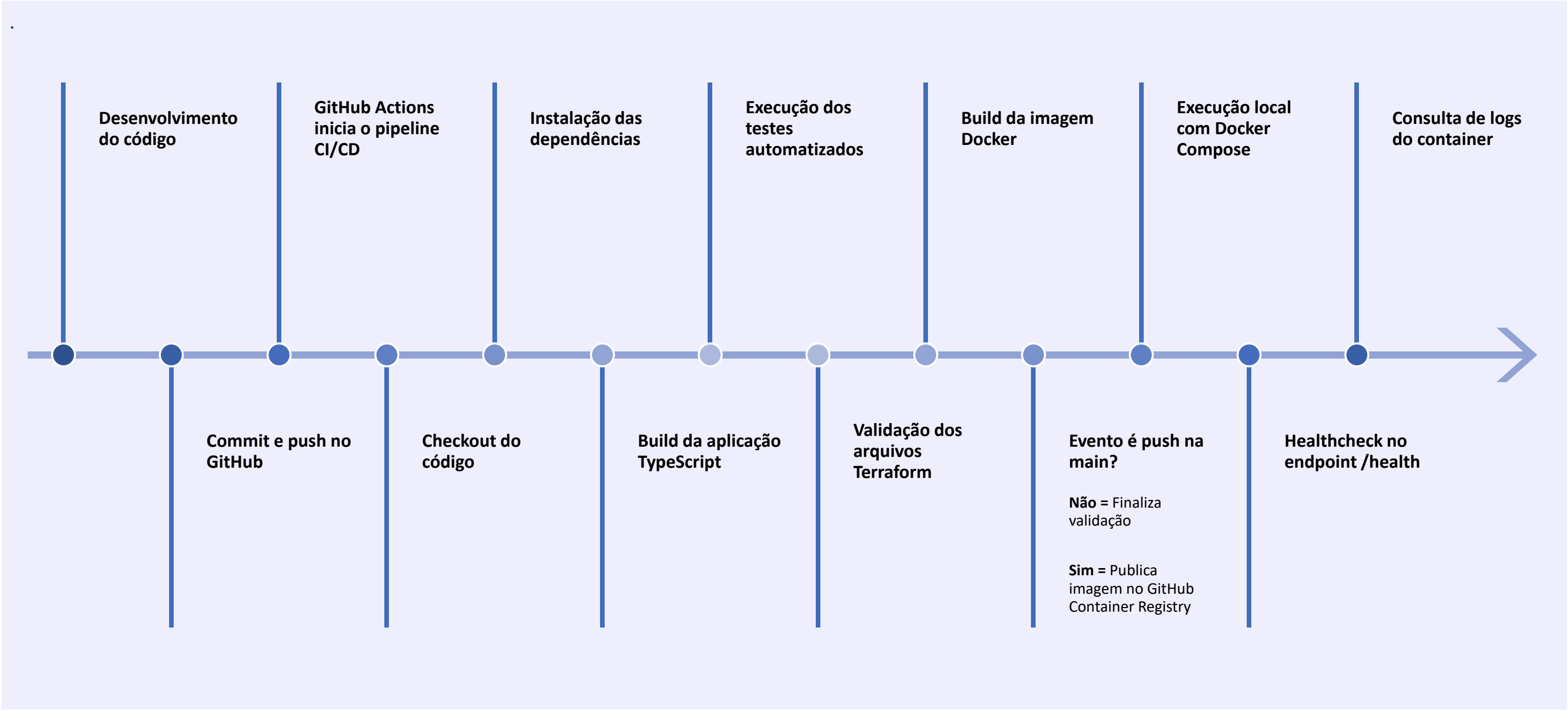
As práticas de segurança incluem ausência de credenciais sensíveis no repositório, uso de secrets no GitHub Actions, separação entre validação de pull request e publicação em main, uso de *.gitignore* e *.dockerignore*, além de imagem final menor, contendo apenas dependências de produção.

Principais entregas da fase 2:

- Pipeline CI/CD.
- Build da imagem Docker.
- Publicação no GitHub Container Registry.
- Dockerfile.
- Docker Compose.
- Scripts de deploy local.
- Healthcheck da aplicação.
- Logs por container.
- Práticas básicas de segurança.
- Relatório final.
- Fluxograma do fluxo DevOps.

b) DEMONSTRAÇÃO PRÁTICA DE TODO O FLUXO DEVOPS IMPLEMENTADO:

Insira aqui a representação visual do Fluxograma completo com a demonstração de todas as etapas do pipeline de integração.



c) ANÁLISE DOS RESULTADOS E SUGESTÃO DE MELHORIAS FUTURAS:

A fase 2 permitiu consolidar um fluxo DevOps mais completo para o projeto.

A aplicação passou a contar com validação automatizada, testes executados no pipeline, empacotamento em container, publicação de imagem Docker e execução local padronizada por Docker Compose ou scripts de deploy.

Como resultado, o processo de entrega tornou-se mais confiável, pois o código é validado antes da publicação da imagem. A containerização também facilita a reprodução do ambiente, reduzindo diferenças entre máquinas e simplificando a execução da aplicação.

O uso do endpoint /health possibilita uma forma simples de monitoramento da aplicação. Os logs do container permitem acompanhar a execução e identificar possíveis falhas. As práticas de segurança aplicadas reduzem o risco de exposição de credenciais e separam o processo de validação em pull requests da publicação em ambiente principal.

Como melhorias futuras, recomenda-se evoluir as práticas já implementadas, adicionando análise automatizada de vulnerabilidades da imagem Docker, centralização de logs em ferramenta de observabilidade, alertas para falhas no healthcheck, ambiente de homologação e deploy automatizado em nuvem.

GRADUAÇÃO
PUCRS online